

University of Tehran

College of Engineering

School of Electrical and Computer Engineering

VLSI Design

Third Computer Assignment

Performance Analysis of Adders and Monte Carlo Simulations

Prepared by:

Faezeh Misaghi

Instructor:

Dr. Vahdat

Academic Year 2025-2026

Contents

1	Part One	2
1.1	Implementation of Each Adder Structure	2
1.1.1	Design and Implementation of the 8-bit Ripple-Carry Adder (RCA)	5
1.1.2	Design and Implementation of the Brent-Kung Adder	6
1.2	Verification of Each Adder's Functional Correctness	7
1.2.1	Automated Test Vector Generation	7
1.2.2	Simulation and Results Analysis	8
1.3	Calculation of Maximum Delay for Each Adder	8
1.4	Calculation of Dynamic Power for Each Adder	9
1.5	Calculation of Static Power for Each Adder	10
2	Part Two	12
2.1	Power Consumption Analysis	13
2.2	Delay Analysis	14
2.3	Delay and Power Trade-off Analysis	14
2.4	Conclusion	15
3	Part Three	15
3.1	Investigation of Vdd Voltage Fluctuation Effects Using Monte Carlo Analysis	15
3.1.1	Analysis of Scatter Plots	16
3.1.2	Statistical Distribution Analysis (Histograms)	17
3.2	Investigation of Temperature Fluctuation Effects	18
3.2.1	Delay Analysis	18
3.2.2	Dynamic Power Analysis	19

1 Part One

1.1 Implementation of Each Adder Structure

The main objective of this project is to design, simulate, and compare the performance of various adders in 90nm CMOS technology. The first step in this project was to create a library of basic logic gates at the transistor level. This library was simulated in the `gates.sp` file and includes subcircuits for the following gates:

- INV
- NAND-2 inputs
- NOR-2 inputs
- AND-2 inputs
- OR-2 inputs
- XOR-2 inputs

According to the project instructions, the channel length (L) for all transistors is set to the minimum allowable value in this technology, namely $L = 100\text{nm}$. To achieve equal resistances in the pull-up and pull-down networks, the width of the PMOS transistor must be chosen larger. Based on the common approximation $\mu_n \approx 2\mu_p$, the following ratio was selected for sizing:

$$\frac{W_p}{W_n} = 2$$

INV

The inverter serves as the base and reference cell for sizing other gates.

- **PDN:** Consists of one NMOS transistor. The base width for this transistor was chosen as $W_n = 180\text{nm}$. The resistance of this path is considered as the reference resistance (R_{ref}).
- **PUN:** Consists of one PMOS transistor. For the resistance of this network to be equal to R_{ref} , given the ratio $W_p/W_n = 2$, its width must be twice that of the NMOS.
- **Final Dimensions:**
 $W_n = 180\text{nm}$
 $W_p = 360\text{nm}$

NAND2

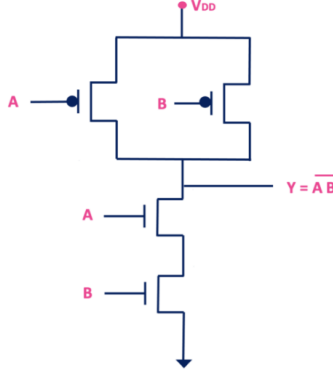


Figure 1: Transistor structure of the NAND2 gate.

- **Structure:** The PDN consists of two series NMOS transistors, and the PUN consists of two parallel PMOS transistors.
- **PDN:** In the worst case, both inputs are '1' and both series NMOS transistors are turned on. The total resistance of the path is equal to the sum of the resistances of the two transistors. For the total resistance of this series path to equal the reference resistance (R_{ref}), the resistance of each NMOS must be half of R_{ref} . To halve the resistance, the channel width must be doubled.
$$W_n = 2 \times W_{n,\text{inv}} = 2 \times 180\text{nm} = 360\text{nm}$$
- **PUN:** In the worst case, only one of the inputs is '0' and only one of the parallel PMOS transistors connects the path to VDD. For this single path to have the equivalent resistance of the reference inverter, its PMOS width must be equal to the PMOS width of the inverter.
$$W_p = 1 \times W_{p,\text{inv}} = 1 \times 360\text{nm} = 360\text{nm}$$
- **Final Dimensions:**
$$W_n = 360\text{nm}$$
$$W_p = 360\text{nm}$$

NOR2

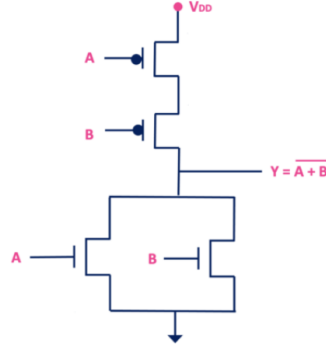


Figure 2: Transistor structure of the NOR2 gate.

- **Structure:** The PDN consists of two parallel NMOS transistors, and the PUN consists of two series PMOS transistors.
- **PDN:** In the worst case, only one of the inputs is '1' and only one of the parallel NMOS transistors is on. For the resistance of this single path to equal R_{ref} , its NMOS width must be identical to the NMOS width of the reference inverter.
$$W_n = 1 \times W_{n,\text{inv}} = 1 \times 180\text{nm} = 180\text{nm}$$
- **PUN:** In the worst case, both inputs are '0' and both series PMOS transistors are on. For the total resistance of this series path to equal the PMOS resistance of the reference inverter, the resistance of each PMOS must be halved. Therefore, their width must be twice the PMOS width of the reference inverter.
$$W_p = 2 \times W_{p,\text{inv}} = 2 \times 360\text{nm} = 720\text{nm}$$
- **Final Dimensions:**
$$W_n = 180\text{nm}$$
$$W_p = 720\text{nm}$$

XOR2

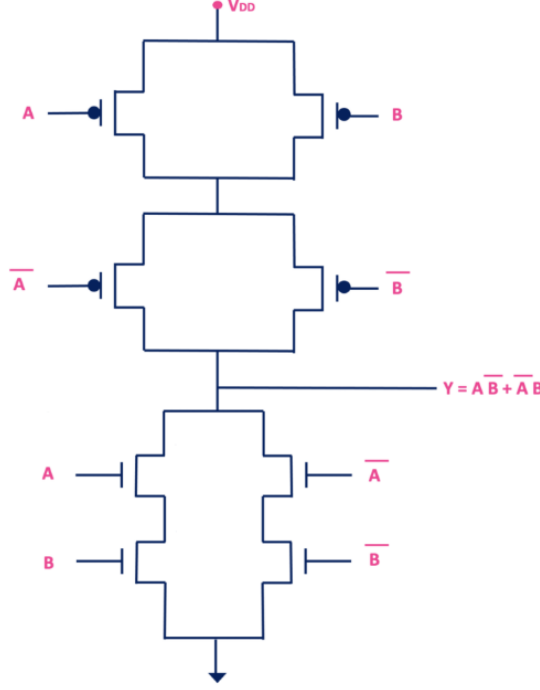


Figure 3: Transistor structure of the 12-transistor XOR2 gate.

Structure: The XOR gate designed in this project is based on a 12-transistor logic CMOS architecture. Two inverters are placed at the input to generate the signals $\bar{A}$ and $\bar{B}$, readying them for the main logic network. The remaining 8 transistors are arranged to implement the function $A \oplus B = \bar{A}B + A\bar{B}$. This network is a combination of series and parallel structures, symmetrically designed so that the correct logic function is maintained in both input states. The transistor sizes in the XOR gate were set to $W_n = 180\text{nm}$ and $W_p = 360\text{nm}$. This choice was made with the goal of balancing area, power consumption, and propagation delay.

1.1.1 Design and Implementation of the 8-bit Ripple-Carry Adder (RCA)

The Ripple-Carry Adder (RCA) is one of the simplest and most intuitive structures for binary addition of multi-bit numbers. The operation of this architecture is based on the serial connection of multiple 1-bit Full Adders.

The primary building block of this adder is a **1-bit Full Adder**. This circuit adds three 1-bit inputs, A , B , and C_{in} , and produces two outputs, Sum and C_{out} . This module was implemented using the gates defined in the `gates.sp` library.

The logical relations governing this circuit are as follows:

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}$$

$$C_{\text{out}} = (A \cdot B) + (C_{\text{in}} \cdot (A \oplus B))$$

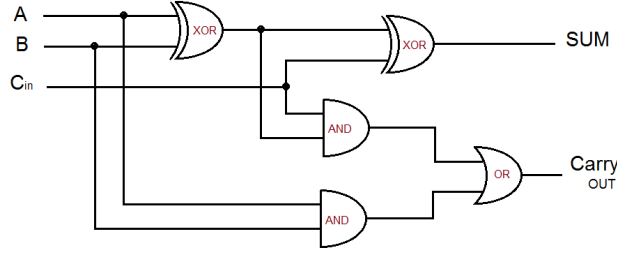


Figure 4: Logical schematic of a Full Adder.

To construct an N -bit adder, it is sufficient to connect N of these full-adder modules together in a chain. To implement the 8-bit adder in this project, 8 full-adder modules were cascaded alongside each other, as shown in the figure.

The most important feature of this architecture is how the carry signal propagates: the C_{out} of each stage is fed as the C_{in} to the next (more significant) stage. This chained connection causes the carry signal to ripple from the least significant bit (LSB) to the most significant bit (MSB), which is why this adder is called a **Ripple-Carry** adder. This serial structure creates a long Critical Path, as calculating the final bits (Sum_7 and C_{out}) depends on the calculation of all intermediate carries.

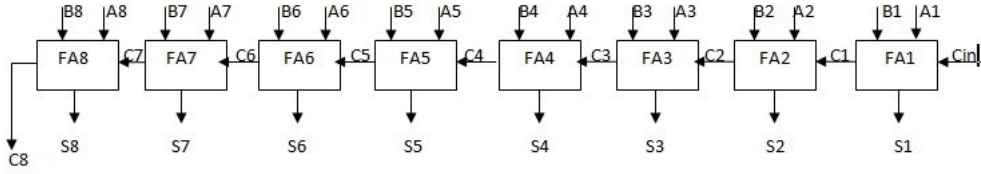


Figure 5: Structural diagram of the 8-bit Ripple-Carry Adder (RCA).

1.1.2 Design and Implementation of the Brent-Kung Adder

The Brent-Kung adder is an advanced architecture from the Parallel Prefix Adder family, designed to overcome the linear delay limitation found in traditional adders like the Ripple-Carry. While the delay in a ripple adder grows linearly ($O(N)$) with the increase in the number of bits (N), the Brent-Kung structure leverages parallel computations to reduce the delay logarithmically ($O(\log N)$). This significant performance improvement is achieved by dividing the addition process into three distinct and efficient stages.

To implement this algorithm modularly, first, in the pre-processing stage, the primary inputs A and B are converted into intermediate Generate and Propagate signals. These signals are calculated according to the following relations and determine whether a given bit independently generates a carry or merely propagates the input carry from the previous bit to the next stage.

$$g_i = A_i \cdot B_i$$

$$p_i = A_i \oplus B_i$$

The heart of this architecture is the Black Cell (BC), which acts as the fundamental building block in the prefix tree. This cell receives two pairs of generate and propagate signals from previous stages and combines them to create a new pair for a larger group of bits. The logic governing this key cell is as follows:

$$g_{\text{out}} = g_1 + (p_1 \cdot g_0)$$

$$p_{\text{out}} = p_1 \cdot p_0$$

The manner in which the carry is calculated and propagated in this structure is the main difference from sequential methods. This parallel process within the circuit's tree structure, depicted in the diagram below using the Black Cell (BC) computational blocks, is best understood as consisting of two main parts: a “Forward Tree” that gathers information from less significant bits to more significant bits, and a “Reverse Tree” that distributes the intermediate results to calculate the final carries. In the forward tree, the initial g and p signals are combined hierarchically so that the carries for larger groups of bits are calculated rapidly. This separation and parallel processing break the linear dependency chain of the carries and significantly shorten the circuit's critical path.

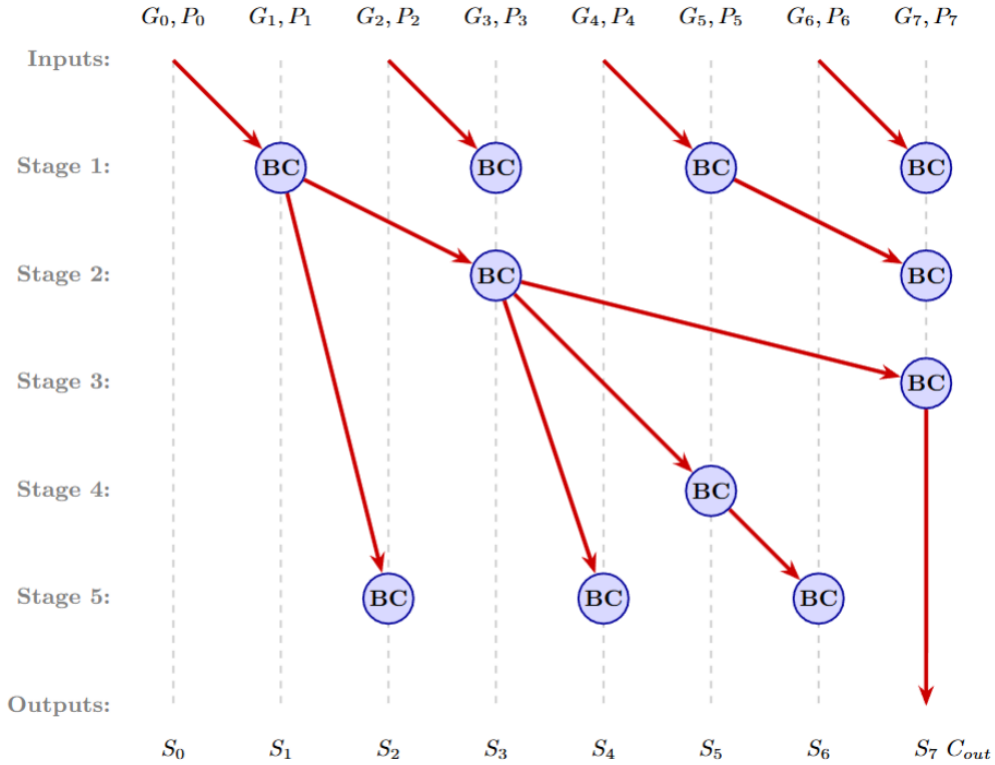


Figure 6: Parallel computation of carries using computational cells (BC)

After generating the group carries in different layers of the tree, these values are combined with the initial signals to obtain the final input carry for each column (C_{i-1}). In the final stage, having all the required carries, the final sum value for all bits is calculated

simultaneously, completely in parallel, according to the following relation:

$$S_i = p_i \oplus C_{i-1}$$

1.2 Verification of Each Adder's Functional Correctness

Upon completing the design, it was essential to ensure the logical correctness of both adders under a wide range of inputs. For this purpose, a comprehensive automated testing method based on a Digital Vector File (DVF) was employed.

1.2.1 Automated Test Vector Generation

To create a comprehensive test suite, a Python script named `vec_generator.py` was written. This script generated a DVF file with 1500 random test cases. The operational logic of this script was as follows:

1. **Generating Random Inputs:** In each iteration, two random 8-bit numbers were generated for inputs A and B, along with a random bit for Cin.
2. **Calculating the Reference Result:** The script performed arithmetic addition to calculate the correct values for outputs S and Cout.
3. **Formatting and Storage:** Finally, all data was stored in the `adder8bit_test.vec` file in standard hexadecimal and binary formats required by HSPICE.

Furthermore, to ensure the functional integrity of the testing mechanism itself, a vector with an intentionally incorrect output was appended to the end of the file. This ensured that HSPICE would detect and report any discrepancies if they occurred.

1.2.2 Simulation and Results Analysis

To execute the test, two separate netlists (`test_rca.sp` and `test_bk.sp`) were created. In each netlist, the `adder8bit_test.vec` vector file was linked to the simulation using the `.vec` command. Then, a transient simulation (`.tran`) was run to apply all 1500 test vectors to each of the adders.

After the simulations concluded, the output error files (`.err`) for both circuits were examined. For both the Ripple-Carry and Brent-Kung adders, the corresponding error files were completely empty. The empty `.err` files conclusively prove that the simulated outputs of the circuits in HSPICE perfectly matched the reference values calculated by the Python script for all 1500 random inputs. This result confidently confirms the logical correctness of both designs.

1.3 Calculation of Maximum Delay for Each Adder

To measure the Worst-Case Delay, the longest signal propagation path, or the critical path, must be stimulated in the circuit. In adder architectures, this situation occurs when C_{in} is forced to ripple from the first bit (LSB) to the last bit (MSB), affecting every single

stage. To realize this scenario, all cells must be placed in a Propagate state. This means the condition $A_i \neq B_i$ must hold for every bit. For this reason, the inputs were selected as follows:

- **Initial Steady State:** $A = 11111111_2$ and $B = 00000000_2$ along with $C_{in} = 0$.
- **At time 10 ns,** the C_{in} input transitions from 0 to 1. This rising edge creates a wave of changes along the computational chain until the outputs finally settle.

To extract the critical path, the circuits were first placed in the initial steady state, and then delays were measured by applying a rising edge to C_{in} under the worst-case scenario. By definition, the maximum propagation delay of the circuit equals the time of the last output to settle (the maximum delay among all outputs).

The results are summarized in the table below:

Table 1: Comparison of propagation delay in the worst-case scenario.

Adder Structure	tpd,Cout	tpd,S8	Maximum Total Circuit Delay (tpd,max)
Ripple Carry Adder (RCA)	551.4 ps	514.0 ps	551.4 ps (Dependent on C_{out})
Brent-Kung (BK)	286.6 ps	384.8 ps	384.8 ps (Dependent on S_8)

- **Ripple Carry Adder (RCA):** In this structure, there is a linear data dependency with a time complexity of $O(n)$. The carry bit of each stage must wait for the previous stage before it can be generated. For this reason, the C_{out} output, which is at the end of this 8-stage chain, settles last and, by recording a delay of 551.4 ps, acts as the primary bottleneck determining the maximum delay of the circuit.
- **Brent-Kung (BK) Adder:** This structure is a Parallel Prefix Adder that breaks the linear dependency of the carry and reduces the time complexity to $O(\log_2 n)$ using a parallel computational tree. The simulation results show that this logistical tree acts highly efficiently, improving the C_{out} generation delay by 48% compared to the RCA, reducing it to 286.6 ps. In this architecture, the last output to settle is the final sum bit (S_8) with a delay of 384.8 ps; this is because, after the rapid computations of the carry tree, the signals must pass through a final layer of XOR gates to generate the sum bits (S).

Final Conclusion: The quantitative simulation proves that the Brent-Kung adder reduced the maximum delay of the total circuit from 551.4 ps to 384.8 ps, improving hardware processing speed by approximately 30%.

1.4 Calculation of Dynamic Power for Each Adder

In this section, the goal is to calculate and compare the dynamic power of the Ripple Carry (RCA) and Brent-Kung (BK) adder structures. To do this, the input vector file (DVF) generated in section (c), comprising 1500 random vectors, was used.

To simulate the circuit near the maximum safe operating frequency, the timing parameters of the DVF file were adjusted based on the critical path delay of each circuit. According to the results, the maximum delay for the RCA adder was 551.4ps, and for the Brent-Kung, it was 384.8ps. Therefore, the period of the input signals was roughly chosen to be 1.5 times these values:

- For RCA:

$$T_{\text{RCA}} \approx 1.5 \times 551.4 \approx 830 \text{ ps}$$

- For BK:

$$T_{\text{BK}} \approx 1.5 \times 384.8 \approx 580 \text{ ps}$$

By executing the transient simulation (`.tran`) for a sufficient duration to cover all vectors, the total average power of the circuits was extracted using the `.measure` command. It should be noted that in the HSPICE environment, the measured total power includes both dynamic and static power components:

$$P_{\text{avg}} = P_{\text{dyn}} + P_{\text{static}}$$

In many applications, if switching is dominant, average power is used as an estimate for dynamic power. However, to achieve a more precise analysis in this project, static power was calculated separately, and then dynamic power was obtained by subtracting static power from the total power.

The obtained results are as follows:

Table 2: Comparison of average power consumption.

Adder Structure	Average Power (W)
Brent-Kung (BK)	2.128×10^{-4}
Ripple Carry (RCA)	1.537×10^{-4}

Using the values obtained from the simulation:

- For Brent-Kung:

$$P_{\text{dyn}} = 2.128 \times 10^{-4} - 1.573 \times 10^{-4} = 5.55 \times 10^{-5} \text{ W}$$

- For RCA:

$$P_{\text{dyn}} = 1.537 \times 10^{-4} - 4.303 \times 10^{-5} = 1.106 \times 10^{-4} \text{ W}$$

The results show that although the total average power is higher in the Brent-Kung structure, a significant portion of it is due to leakage power, and its dynamic power is actually less than that of the Ripple Carry. Conversely, in the RCA structure, the share of dynamic power is more dominant, which stems from the ripple propagation and sequential switching along the carry path.

1.5 Calculation of Static Power for Each Adder

In this section, the static power of both the Ripple Carry and Brent-Kung adders was examined. Static power refers to the power consumption of a circuit when the inputs are constant and no switching occurs. This power is primarily due to transistor leakage currents in CMOS technology.

To accurately extract static power, a sampling time interval was chosen such that the circuit was in a steady state. To achieve this, an interval was selected in each period following the *tdelay* time (when the outputs settle) and before the start of the next cycle. This interval lacks any changes in inputs and outputs, providing suitable conditions for measuring leakage power.

Using the `.measure` command and selecting the appropriate time interval, the static power for both circuits was extracted. The results are provided in Table 3.

Table 3: Comparison of static power consumption.

Adder Structure	Static Power (W)
Brent-Kung (BK)	1.573×10^{-4}
Ripple Carry (RCA)	4.303×10^{-5}

Overall, the results indicate that although the Brent-Kung adder excels in terms of operating speed, this improvement comes at the cost of increased static power. On the contrary, the Ripple Carry structure has lower static power, but its dynamic power is higher.

This behavior reflects a crucial trade-off in digital circuit design: in Brent-Kung, due to the use of a larger number of gates and transistors, leakage currents increase, leading to higher static power. Meanwhile, the Ripple Carry structure has higher switching activity due to the sequential propagation of the carry signal, resulting in higher dynamic power.

Therefore, the choice between these two structures depends on the desired design criteria: if speed is the priority, Brent-Kung is the more suitable option, whereas in power-sensitive applications, especially regarding leakage power, the Ripple Carry structure might be a better choice.

Table 4: Overall comparison of power in both adder structures.

Structure	Average Power (W)	Dynamic Power (W)	Static Power (W)
Brent-Kung (BK)	2.128×10^{-4}	5.55×10^{-5}	1.573×10^{-4}
Ripple Carry (RCA)	1.537×10^{-4}	1.106×10^{-4}	4.303×10^{-5}

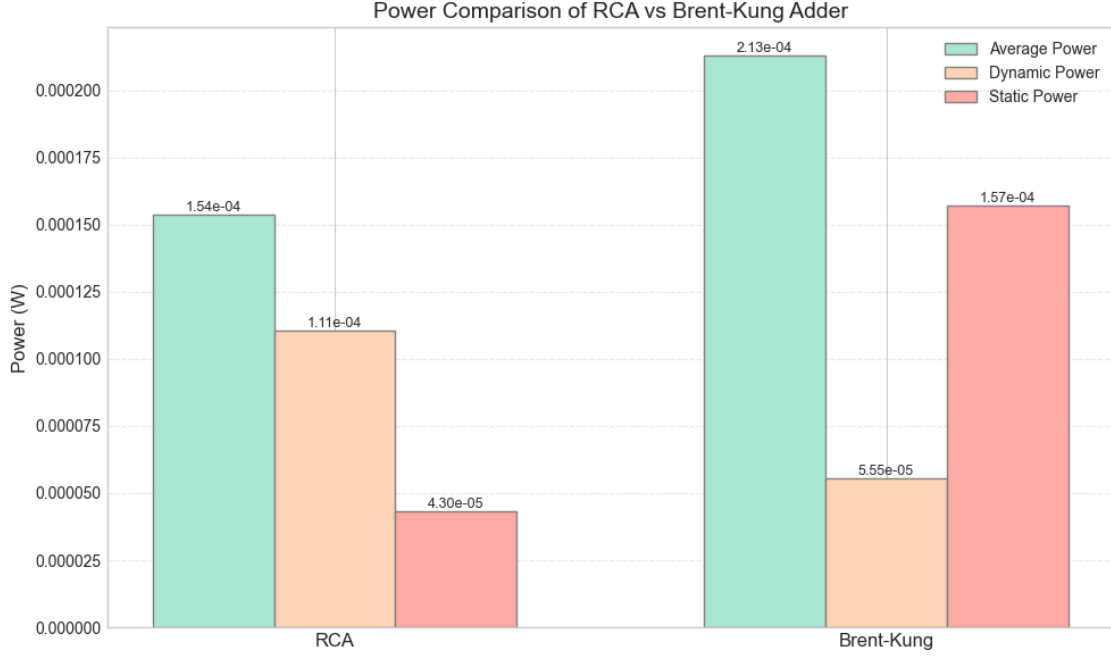


Figure 7: Overall power comparison chart.

2 Part Two

In this section, the performance of the Brent-Kung adder under the influence of manufacturing process variations across five different process corners was investigated using simulation in the `bk_corners_sim.sp` file. For each process corner, key parameters including maximum propagation delay, total power consumption, static power consumption, and dynamic power consumption were extracted. Dynamic power in this analysis is derived from the difference between total power and static power.

The simulation results are summarized in Table 5:

Table 5: Summary of simulation results at various process corners (Vector Analysis).

Corner	$t_{pd,max}$ (ps)	P_{total} (μ W)	P_{static} (μ W)	$P_{dynamic}$ (μ W)
TT	141.3	1.313	0.323	0.991
FF	110.9	5.468	4.968	0.500
SS	181.4	1.121	0.046	1.074
FS	136.6	2.533	1.728	0.805
SF	150.2	1.479	0.412	1.067

Based on the above results, the worst-case scenario for the different parameters is as follows:

- The worst-case propagation delay corresponds to the SS corner with a value of 181.4 ps, because in this state both NMOS and PMOS transistors are slow and drive current is severely reduced.
- The worst-case dynamic power corresponds to the SS corner with a value of $1.074 \mu\text{W}$, which occurs due to longer signal rise/fall times and the resulting increase in short-circuit current in the gates.
- The worst-case static power corresponds to the FF corner with a value of $4.968 \mu\text{W}$, driven by the reduced threshold voltage of transistors and the sharp, exponential increase in Subthreshold Leakage current at this fast process corner.

To better understand the circuit's behavior, Figures 8 through 10 have been plotted.

2.1 Power Consumption Analysis

As observed in Figure 8, the variations in static power between the different corners are quite extensive; in the FF corner, static power spikes dramatically, constituting the majority of the circuit's total power. Conversely, the SS corner records the lowest amount of static and leakage power due to a higher threshold voltage. Influenced by switching voltages and gate capacitance characteristics, dynamic power displays higher values in the SS and SF corners.

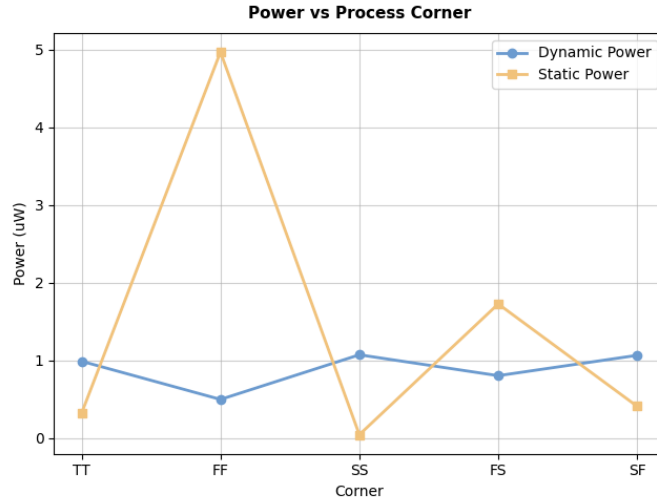


Figure 8: Power consumption analysis at various process corners.

2.2 Delay Analysis

According to Figure 9, the lowest delay and highest speed belong to the FF corner at 110.9ps, while the highest delay belongs to the SS corner at 181.4ps. This behavior is entirely consistent with manufacturing process theories, reflecting the variations in transistor current drive when moving between fast and slow corners. The unmatched FS and SF corners exhibit intermediate behaviors.

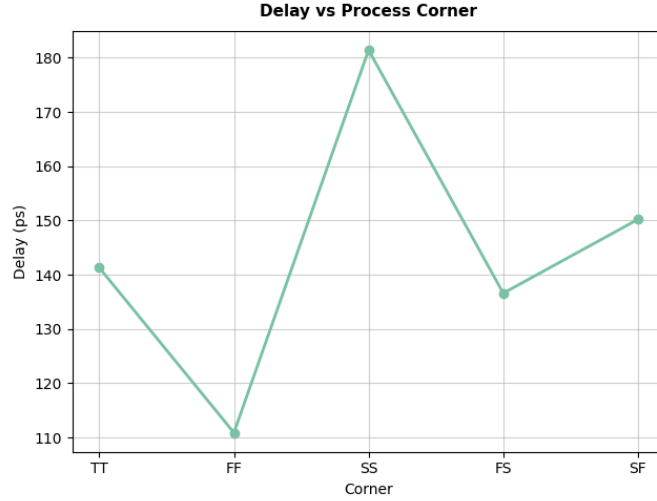


Figure 9: Maximum delay analysis at various process corners.

2.3 Delay and Power Trade-off Analysis

Figure 10 depicts the relationship between delay and total power consumption, illustrating the classic speed-power trade-off in VLSI design. Although the FF corner pushes the circuit into its fastest possible state, it imposes a heavy penalty in the form of high power consumption ($5.468 \mu\text{W}$). On the opposite end, the SS corner consumes the least total power by sacrificing speed. The other corners fill the design space between these two extreme conditions.

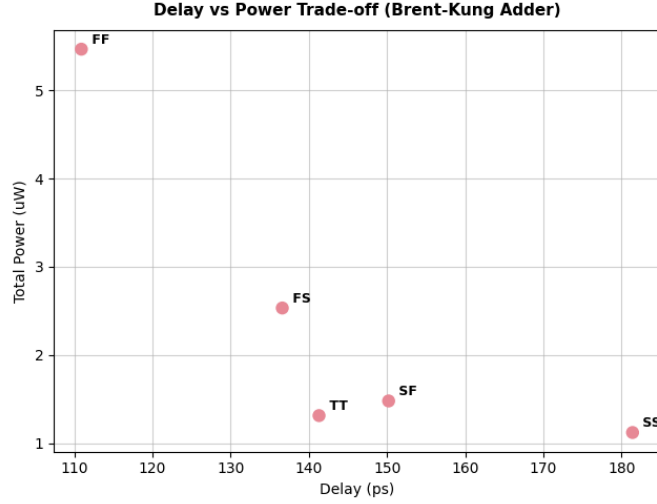


Figure 10: Trade-off chart between maximum delay and total power.

2.4 Conclusion

The results indicate that no single process corner offers the best performance across all metrics. Fast corners are suitable for applications requiring high speed but are accompanied by increased power consumption. Conversely, slow corners consume less power but face delay limitations. Therefore, selecting the optimal operating point depends on the specific design requirements and the final application of the circuit.

3 Part Three

3.1 Investigation of Vdd Voltage Fluctuation Effects Using Monte Carlo Analysis

In this section, a statistical Monte Carlo analysis was performed to investigate the impact of power supply voltage fluctuations on the performance of the Brent-Kung adder. The simulations were implemented in the `monte_carlo_sim.sp` file, and the output results were extracted, processed, and plotted using a Python script named `plot_generator.py`.

To model voltage variations, instead of using an ideal analytical distribution, a DVF input file was employed to generate voltage samples based on a specific statistical distribution. This approach ensures the circuit's behavior is analyzed under conditions closer to actual noise and process variations. The supply voltage in each Monte Carlo run was set equal to the value sampled from this distribution.

The Monte Carlo simulation was executed for 100 iterations to evaluate the statistical behavior of the circuit in terms of power and delay.

Also, similar to the previous section, dynamic power was calculated from the difference between total power and static power:

$$P_{\text{dyn}} = P_{\text{total}} - P_{\text{static}}$$

Ideally, the theoretical relation for dynamic power is:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V_{DD}^2 \cdot f$$

However, in this simulation, due to the use of sampled data as well as temporal and non-ideal circuit effects, the observed behavior might deviate from this simple model.

3.1.1 Analysis of Scatter Plots

- **Dynamic Power vs. VDD**

The scatter plot of dynamic power versus supply voltage illustrates a non-linear dependence between these two quantities. Although a quadratic relationship is expected based on the theoretical model, simulation results indicate that this dependence is not completely ideal in practice.

This is because the measured power is calculated as a time average and, in addition to being voltage-dependent, is affected by parameters such as switching time, signal activity, and the circuit's dynamic variations. Particularly as VDD increases, the reduction in switching delay can lead to decreased energy consumption time, somewhat moderating the quadratic growth effect of power.

Therefore, observing a deviation from a perfectly quadratic behavior signifies that more realistic effects are being captured in the simulation.

- **Critical Path Delay vs. VDD**

In this chart, the inverse relationship between delay and voltage is clearly visible. As voltage increases, transistor drive current rises and switching speed increases, resulting in decreased delay.

At lower voltages, especially near the threshold voltage of transistors (V_t), the delay increases sharply and non-linearly. This behavior originates from the significant reduction in drive current in this region, which heavily slows down the circuit's performance.

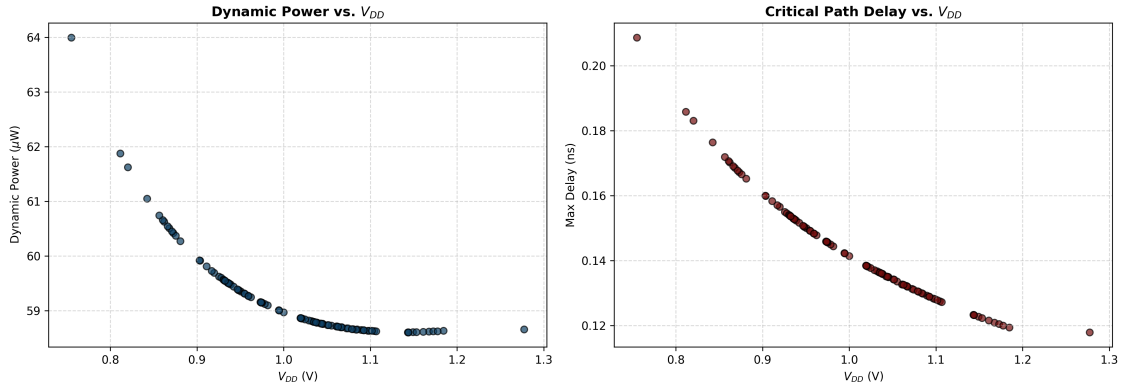


Figure 11: Scatter plots of dynamic power and critical path delay versus supply voltage.

3.1.2 Statistical Distribution Analysis (Histograms)

- **Dynamic Power Distribution**

The histogram for dynamic power resembles a normal distribution, but it is not perfectly symmetrical and exhibits right skewness.

This phenomenon occurs due to the non-linear relationship of power with voltage as well as the nature of the input data (DVF samples). Since dynamic power is roughly proportional to the square of the voltage, positive voltage deviations have a larger impact than negative deviations. As a result:

- The probability of encountering larger power values increases.
- The tail of the distribution stretches to the right.

Therefore, even if the input distribution is close to Gaussian, the system's output deviates from an ideal Gaussian state due to non-linearity.

- **Critical Path Delay Distribution**

The delay histogram exhibits severe right skewness, which is a well-known characteristic in the statistical analysis of VLSI circuits.

Delay has a strong non-linear dependency on voltage and roughly follows this relation:

$$\text{Delay} \propto \frac{V_{DD}}{(V_{DD} - V_t)^\alpha}$$

As a result:

- A reduction in voltage causes a severe increase in delay.
- An increase in voltage provides only a limited improvement in delay.

This asymmetry produces a small number of samples with very high delays, creating a long tail on the right side of the distribution. This tail plays a key role in determining the worst-case delay and consequently the circuit's Timing Yield.

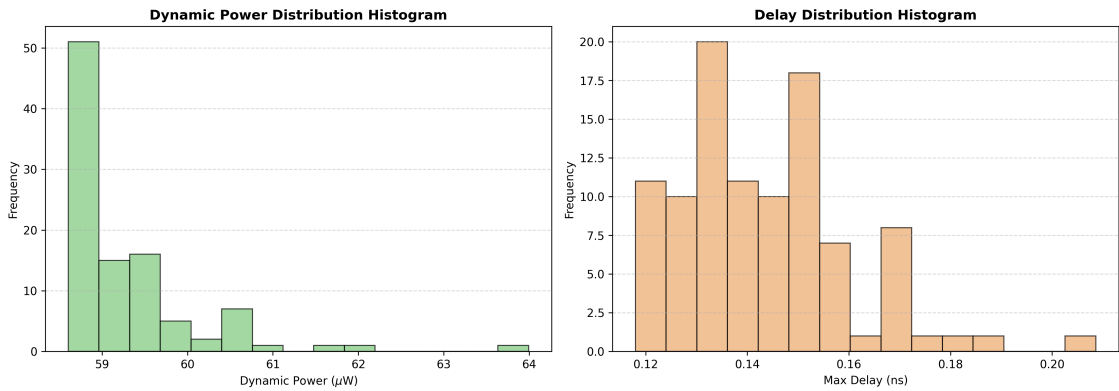


Figure 12: Histograms for the distribution of dynamic power and critical path delay.

The results of this analysis show that:

- Circuit behavior under real voltage fluctuations (modeled by DVF) does not necessarily strictly adhere to ideal models.
- Dynamic power has a non-linear dependence on voltage, but timing and dynamic effects can moderate this dependence.
- Delay is highly sensitive in the low-voltage region.
- Output distributions are not perfectly Gaussian and symmetrical; they contain skewness and tails.
- Worst-case metrics are influenced more by these tails than by the mean value.

3.2 Investigation of Temperature Fluctuation Effects

In this section, to examine the effect of temperature on circuit performance, a simulation file named `temperature_sim.sp` was designed and executed. In this file, using the `sweep` command in the HSPICE environment, the behavior of the circuit across a range of different temperatures was analyzed.

The inputs to the adder were not directly assigned values; instead, they were applied to the circuit via an external file. This method ensures that the stimulus conditions are closer to reality and allows for the application of diverse test vectors.

After executing the simulation, the outputs were processed using a Python script to enable the plotting of graphs and precise data analysis.

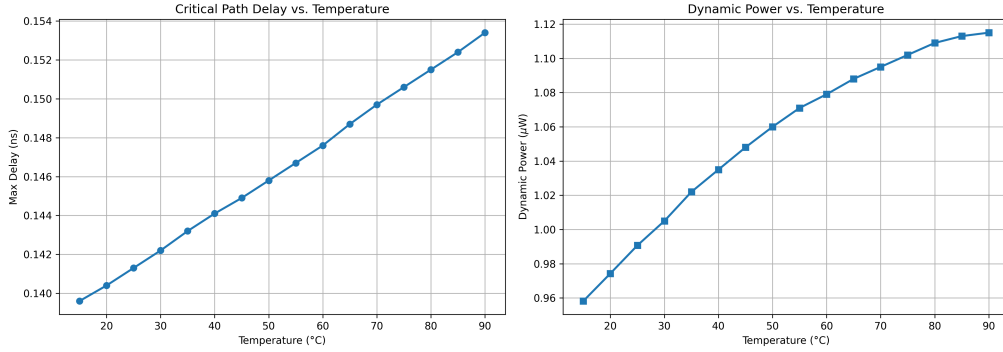


Figure 13: Plot of delay and dynamic power versus temperature.

3.2.1 Delay Analysis

The results indicate that as temperature increases, the critical path delay increases uniformly and almost linearly. The delay value rose from approximately 0.139 ns at 15°C to about 0.153 ns at 90°C.

This behavior is completely consistent with the physical principles of CMOS transistors. As temperature increases, carrier mobility in the transistor channel decreases, which leads to a reduction in drive current. Consequently, switching speed decreases and delay increases.

Furthermore, the linear trend of these changes indicates stable circuit operation within this temperature range, without entering non-linear or critical regions.

3.2.2 Dynamic Power Analysis

Regarding dynamic power, the results show a mild increase as temperature rises. Dynamic power grew from approximately $0.96 \mu\text{W}$ to about $1.115 \mu\text{W}$.

Although temperature does not appear directly in the classical dynamic power relation ($P = \alpha CV^2f$), its indirect effects can be observed through increased leakage current, changes in switching timing, and the potential increase in glitches.

The increasing yet controlled trend in power shows that the circuit exhibits stable behavior concerning energy consumption against temperature variations and does not suffer from thermal runaway.